

Intermediate (wrap real logic behind a route) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

`run_chat()` below is a small stand-in for Doc04's real chat function — swap in the real one, the route around it doesn't change. Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

def run_chat(message):
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output (`curl -X POST localhost:8000/chat -d '{"message": "hi"}'`):

```
{"reply": "You said: hi"}
```

Expected output for a missing `message` field:

422 Unprocessable Entity

This meets the exercise's core requirement — typed request in, typed response out, errors don't crash the server. It treats every internal failure as a 500 with no distinction from a bad input, and it's missing type hints on the route function itself — both addressed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — splitting 400 (bad input) from 500 (real failure)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    """Raised on purpose when input is well-formed but not usable."""
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output for `{"message": ""}` :

```
400 {"detail": "message cannot be empty"}
```

Expected output for `{"message": "hi"}` :

```
200 {"reply": "You said: hi"}
```

Approach 2 — moving the models into `schemas.py`

Splitting request/response models into their own file is the layout the Build Task's `Suggested files` section uses — worth practicing here first, on something small.

```
# schemas.py
from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str
```

```
# main.py
from fastapi import FastAPI, HTTPException
from schemas import ChatRequest, ChatResponse

app = FastAPI()

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        return ChatResponse(reply=run_chat(request.message))
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output: identical to Approach 1 — this is purely a file-organization change.

Difference from Basic: both Intermediate approaches add a dedicated `ChatInputError` so a deliberate, expected rejection (empty message) returns 400 instead of being lumped in with genuine internal failures at 500 — a client should be able to tell "fix your request" apart from "try again later,

it's on us." Approach 2 additionally separates the Pydantic models into `schemas.py`, keeping `main.py` focused on routing logic only.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — logging the real error, leaking nothing to the client

```
import logging
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    if message == "boom":
        raise RuntimeError("simulated internal failure")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        return ChatResponse(reply=run_chat(request.message))
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        logger.exception("chat route failed")
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected client-facing output for `{"message": "boom"}` :

```
500 {"detail":"Something went wrong."}
```

Expected server terminal output for the same request (this is what `logger.exception()` writes — the full traceback, only ever visible server-side):

```
ERROR:__main__:chat route failed
Traceback (most recent call last):
...
RuntimeError: simulated internal failure
```

The client never sees the word "RuntimeError," the string "boom" , or a stack trace — only the generic message. That gap between what the log shows and what the response shows is the entire point of this exercise's "no raw error trace" requirement.

Approach 2 — a request ID on every log line

This is the same idea the Build Task's middleware requires, shown small: tag every log line from one request with the same ID, so you can find every line that belongs to one failed call, even under concurrent traffic.

```
import logging
import uuid
from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel

logging.basicConfig(level=logging.INFO, format="%(asctime)s [%s] %(message)s",
                    request_id=[request_id])
logger = logging.getLogger(__name__)
app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    if message == "boom":
        raise RuntimeError("simulated internal failure")
    return "You said: " + message
```

```

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest, http_request: Request) -> ChatResponse:
    request_id = http_request.state.request_id
    try:
        reply_text = run_chat(request.message)
        logger.info("chat ok", extra={"request_id": request_id})
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        logger.exception("chat route failed", extra={"request_id": request_id})
        raise HTTPException(status_code=500, detail="Something went wrong.")

```

Expected output: the client sees the same 400/500/200 replies as Approach 1, plus an `X-Request-ID` response header; every server log line for that request now carries the same short ID, so grepping the logs for one ID shows exactly that request's whole story.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate already tells 400 from 500 correctly, but a 500 there gives you no record of *what* actually broke — you'd only know a client got a generic message. Approach 1 closes that gap: the real exception is logged in full, server-side only, never in the client reply. Approach 2 builds on Approach 1 by tagging every log line with a per-request ID, so a specific failed request can be traced through the logs even when many requests are happening at once — this is exactly what the Build Task's "tagged per-request" logging requirement asks for.

Which one should you actually write? Intermediate Approach 1's 400/500 split is the minimum any real API needs — write that first. Add Advanced Approach 1's `logger.exception()` the moment your route can fail in a way you didn't anticipate (which is always, eventually) — silently returning "Something went wrong." with no server-side record is worse than a stack trace nobody sees. Reach for Approach 2's request-ID tagging once your service gets real concurrent traffic and "which of these 500 log lines belongs to which failed request" becomes a real question — which it will, by the time you build the Build Task below.